

Assignment 2-Case Study

Stage 4 Lab Activities

Implementing the SmartCare Domain Layer

DESIGN FIRST -> AI PAIR PROGRAMMING -> REVIEW -> VERIFY | 1hour

A - Revisit Approved UML

Confirm responsibilities, attributes and relationships before coding.

Item	Approval
Responsibilities	All classes defined the responsibilities and doing it properly
Attributes	All the attributes in each class are correct
Relationships	Associations, inheritance, composition are correctly presented

B - Implement Patient: AI OFF

Implement Patient with type hints and basic validation.

```
class Patient:
    def __init__(self, patient_id: int, name: str):
        if patient_id <= 0:
            raise ValueError("Patient ID must be positive")

        if not name.strip():
            raise ValueError("Patient name cannot be empty")

        self.patient_id: int = patient_id
        self.name: str = name
```

C - Implement Practitioner: AI OFF

Implement Practitioner with identifier, name and specialty; no database logic.

```
class Practitioner:
    def __init__(self, identifier: int, name: str, specialty: str):
        self.identifier = identifier
        self.name = name
        self.specialty = specialty
```

D - Implement Appointment: AI ON

Give AI the approved Appointment UML, business rules and explicit constraints. Ask it to implement only Appointment and agreed enum/exception.

Act as a Python pair programmer. Implement only the `Appointment` class from the approved SmartCare UML.

Requirements:

- Follow the approved UML for the Appointment class.
- Use Python type hints.
- Use an `AppointmentStatus` enum for appointment status.
- An appointment is initially `SCHEDULED`.
- A scheduled appointment can be changed to `CANCELLED`.
- Cancelled appointments remain as objects and must not be deleted.
- Protect the appointment status from invalid state changes.
- Implement only the agreed Appointment class, enum, and any agreed exception required for validation.

Constraints:

- Do not add database functionality.
- Do not add a user interface.
- Do not add notification classes.
- Do not add service or manager classes.
- Do not add functionality that is not present in the approved UML or business rules.
- Explain any design decision that is not directly visible in the UML.

E - Review Generated Code

Check model consistency, unsupported features, public state mutation, unnecessary inheritance, invented dependencies and error handling.

Model consistency:	The appointment class allows the approved UML
Unsupported features:	No unrelated service functions were added.
Public State Mutation:	Appointment state is protected so that it cannot be changed without specific rules.
Unnecessary inheritance:	Appointment does not inherit from any unsupported class.
Invented Dependencies:	No unnecessary classes were introduced
Error handling:	Invalid values and appointment status transitions were handled using appropriate validation and exceptions.

F - Manual Behaviour Checks

Create valid objects, test invalid input, cancel a scheduled appointment and attempt an illegal repeated transition

```

# Create valid objects

patient = Patient(1, "John Smith")
practitioner = Practitioner(101, "Dr Sarah Lee", "General Practice")

appointment = Appointment(
    patient=patient,
    practitioner=practitioner,
    appointment_time="2026-10-01 10:00"
)

print(appointment.status)
# Expected: SCHEDULED

# Test invalid input
try:
    invalid_patient = Patient(0, "")
except ValueError as e:
    print(e)
# Expected: ValueError

# Cancel a scheduled appointment
appointment.cancel()

print(appointment.status)
# Expected: CANCELLED

# Attempt an illegal repeated transition
try:
    appointment.cancel()
except ValueError as e:
    print(e)
# Expected: Error because the appointment is already CANCELLED

```

G - Refactor

Remove unnecessary code and make implementation simpler and design-consistent.

- The implementation was reviewed and all unnecessary code were removed to make it simpler and design consistent.

H - AI Engineering Log

Record prompt, generated contribution, decisions and verification evidence.

- Prompt: Complete the implementation of the appointment
- Generated contribution: The AI generated the initial Python implementation of the `Appointment` class
- Decisions: I reviewed the code and made any necessary amendments.

- Verification evidence: I manually tested the implementation by creating valid objects, testing invalid input, cancelling a scheduled appointment, and attempting an illegal repeated cancellation. The expected behaviour was confirmed.

Suggested AI prompt

Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

```
- from enum import Enum
- from datetime import datetime
-
-
- class AppointmentStatus(Enum):
-     SCHEDULED = "SCHEDULED"
-     CANCELLED = "CANCELLED"
-
-
- class Appointment:
-     def __init__(
-         self,
-         appointment_id: int,
-         patient: "Patient",
-         practitioner: "Practitioner",
-         appointment_time: datetime
-     ):
-         self.appointment_id = appointment_id
-         self.patient = patient
-         self.practitioner = practitioner
-         self.appointment_time = appointment_time
-         self._status = AppointmentStatus.SCHEDULED
-
-     @property
-     def status(self) -> AppointmentStatus:
-         return self._status
-
-     def cancel(self) -> None:
-         if self._status == AppointmentStatus.CANCELLED:
-             raise ValueError("Appointment is already cancelled.")
-
-         self._status = AppointmentStatus.CANCELLED
```

Reflection

Which AI-generated part did you modify or reject? Why? How did the approved design constrain the AI?

- Throughout the code, I modified the AI-generated implementation by deleting any unnecessary code that is not available in the approved UML. I made sure that the appointment status could not be changed directly by anyone. Also an appointment could not be cancelled more than once.

The approved design was constraining the AI by limiting the implementation of the classes, attributes, relationships, and responsibilities that was defined in the UML. I did not allow the AI to make modification by adding database logic, notification services, user-interface code, or unnecessary inheritance. This helped me to keep the implementation consistent with the original design.